

Algoritmer

- I STL finns många användbara algoritmer
- Samtliga algoritmer opererar på iteratorer
- Iteratorer finns för alla behållare och algoritmerna fungerar därför på alla behållare
- Detta gäller även dem man skriver själv, förutsatt att iteratorn uppfyller villkoren

Algoritmer

```
#include <algorithm> // algoritmer

int a[] = {1, 4, 2, 8, 5, 7};
const int n = sizeof(a) / sizeof(int); // antal element
std::sort(a, a + n); // sortera
std::copy(a, a + n, // mata ut
          std::ostream_iterator<int>(std::cout, " "));

std::vector<int> b;
b.push_back(7);
...
std::sort(b.begin(), b.end()); // sortera
std::unique(b.begin(), b.end()); // ta bort dubletter
```

Funktionspekare

- Man skapa pekare till funktioner. Vissa algoritmer tar funktionspekare som argument.

```
#include <cstdio> // in och utmatning i C

int foo(int i) {
    printf("i is %d \n", i);
}

int main() {
    int (*fp)(int) = &foo;
    fp(3); // skriver ut "i is 3"

    int (*fp_again)(int) = foo;
    (*fp_again)(4); // skriver ut "i is 4"
}
```

Funktionsmallar

Ett mer avancerat exempel på funktionsmall

```
struct A { virtual ~A() {} };
struct B : A {};
struct C : A {};

template<class T> // generisk factoryfunktion
A *create() // returnerar pekare till basklass
{
    return new T; // skapa instans av typ T
}

A* (*fp)(); // funktionspek, inga arg, returnera A*

fp = create<B>; // peka på B-factory
A *b = fp(); // skapa ett B-objekt

fp = create<C>; // peka på C-factory
A *c = fp(); // skapa ett C-objekt
```

Pekare till medlemsfunktioner

- Precis på samma sätt som man pekar på globala funktioner kan man peka på funktioner i klasser
- Dessa funktionspekare är lite speciella eftersom de måste ha tillgång till en **this**-pekare
- Pekare till medlemsfunktion är egentligen en offset som pekar ut den funktion som ska köras

Pekare till medlemsfunktioner

C++ inför tre nya operatorer för medlemspekare `::*` `->*` och `.*`

```
class A {
public:
    int foo(int i) { cout << "i = " << i << endl; };
};

int main() {
    A a;
    A * apek = &a;

    int (A::*mp)(int); // deklarerar medlemspekare
    mp = &A::foo;      // sätta mp att peka på foo

    (a.*mp)(4);         // kalla på funktionen med operator .*
    (apek->*mp) (3);     // kalla på funktionen med operator ->*
```

Funktionsobjekt

- **Funktionsobjekt** (*function objekt*) är objekt som imiterar syntaxen hos en funktion
- En fördel i jämförelsen med funktionspekare är att funktionsobjekt kan innehålla data
- När man använder funktionspekare sker alla anrop genom att avreferera pekaren. Samtidigt är det svårt att göra funktionen **inline**

Funktionsobjekt

```
struct less_abs
{
    bool operator()(double x, double y)
    { return fabs(x) < fabs(y); }
};

std::vector<double> v;
...

// konstruera less_abs-objekt
std::sort(v.begin(), v.end(), less_abs());

// rand är fkt.pekare
std::vector<int> u(100);
std::generate(u.begin(), u.end(), rand);
```

Funktionsobjekt

Skilj på typer och objekt (instanser av typer)

```
struct A { int i; };
struct Comp {
    bool operator()(const A &a, const A &b) const
    {
        return a.i < b.i;
    }
};

// Comp är en typ och används inuti std::map
// vid kompileringstillfället
std::map<A, A, Comp> m;

// Comp() är en instans och används i funktionen
// std::sort vid körningstillfället
std::vector<A> v;
std::sort(v.begin(), v.end(), Comp());
```

Kapitel 5 - Strömmar

- Utmatning med `std::ostream` och `std::ostringstream`
- Inmatning med `std::istream` och `std::istringstream`
- Överlagring av in- och utmatning
- Iteratorer för in- och utmatning
- Filhantering med strömmar

Strömmar

Exempel på strömmar:

```
#include <iostream> // innehåller std::cout
#include <iomanip>    // innehåller setw()

std::cout << "Demo:" // utmatning av char *
           << std::setw(10) // sätt storlek på utmatning
           << 4711 // utmatning av int
           << std::endl; // radmatning

std::cout << "Name\tPts\n" // rubriker
           << "Joe\t37\n" // '\t' är tabulatortecken
           << "Anna\t41\n"; // '\n' är radmatning

int i, j;
std::cin >> i >> j; // hämta int från stdin
```

Utmatning

Utmatning med
`std::cout`,
`std::cerr`,
`std::clog`

Utmatning med `cout`, `cerr`

- Prefixet `c` står för *character*
- För utmatning används `std::cout` (skriver till `stdout`)
- För felutmatning och loggning använder man `std::cerr` resp. `std::clog` (skriver till `stderr`)
- Samtliga är globala objekt av typen `std::ostream` med överlagrad **operator<<**

Utmatning med `cout`, `cerr`

Den överlagrade operatör << klarar samtliga inbyggda datatyper.

```
std::string str = "C++";
const void *p = &str;
char c = 'a';

std::cout << (int)c << " " // '97' (int)
<< 3.14 << " " // 3.14 (double)
<< str << " " // C++ (std::string)
<< str.c_str() << " " // C++ (const char *)
<< p << std::endl; // 0012FF60 (address)
std::cerr << "could not open file" // stderr
<< std::endl;
```

Formatväljare för utmatning

Man kan variera formatet på sin utmatning:

```
double balance = 147.5371; // pengar på kontot
std::cout.precision(5); // använd fem värdesiffror
std::cout << "Balance: " // rubrik
<< std::setw(10) // tio siffrors fält
<< std::setfill('*') // fyll tomrum med *
<< balance // utskrift ger resultat:
<< std::endl; // Balance: ****147.54
```

Utmatning till / inmatning från minne

Om man vill skriva till / från minnet kan man använda `stringstream` som skriver till / från en STL-sträng

```
#include <stringstream>

std::ostringstream message; // mata till minne
int i = 17;
double d = 33.3;
message << "Error: " << i // fyll med data
<< " is less than " << d;
std::cerr << message.str() << std::endl; // skriv ut

std::istringstream source("To be or not to be");
std::string str;
source >> str; // ett ord i taget
```

Inmatning med `cin`

- För inmatning från `stdin` använder man `cin`
- Typen på argumentet till `operator>>` bestämmer hur indata tolkas

```
// antag inmatning för ett bankkonto:  
// kalle qwerty 147.30  
std::string user, passwd;  
double balance;  
  
std::cin >> user >> passwd >> balance;  
if(passwd != "qwerty")  
    std::cout << "wrong password" << std::endl;  
else  
    std::cout << "balance: " << balance << std::endl;
```

Inmatning med `cin`

För att förhindra att man skriver utanför avsett minne kan man använda formatväljare för `cin`:

```
char buf[10];  
  
// läs 9 tecken + '\0' i varje varv  
while(std::cin >> std::setw(sizeof buf) >> buf)  
    do_something(buf);
```

Varning: initieringsordning hos globala objekt

- Standarden säger inget om initieringsordningen hos objekt som ligger i olika översättningsenheter (.cpp-filer)
- Detta innebär att `std::cout` kan initieras *efter* dina globala objekt och statiska klassmedlemmar
- Programmet får då ett oväntat beteende
- Använd därför aldrig `std::cout` i konstruktorn för globala objekt

Överlagring

Inmatnings- och
utmatningsoperatorer

Överlagring av in- och utmatningsoperatorer

- När man vill överföra data från en ström till ett objekt överlagrar man **operator>>**
- När man vill skriva ut ett objekt till en ström överlagrar man **operator<<**

Överlagring av in- och utmatningsoperatorer

```
// i headerfilen:
class A
{
public:
    int      get_i() const { return i; }
    std::string get_s() const { return s; }
    void set_i(int j)      { i = j; }
    void set_s(std::string t) { s = t; }
private:
    int i;
    std::string s;
};

// deklarationer
std::ostream &operator<<(std::ostream &, const A &);
std::istream &operator>>(std::istream &, A &);
```

Överlagring av in- och utmatningsoperatorer

```
// i implementationsfilen:
std::ostream &
operator<<(std::ostream &o, const A &a) // definition
{
    return o << "{" << a.get_i() << ", "
               << a.get_s() << "}";
}

std::istream &
operator>>(std::istream &o, A &a) // definition
{
    int i;
    std::string s;
    o >> i >> s;
    a.set_i(i);
    a.set_s(s);
    return o;
}

A a;
// skriv till objektet från stdin
std::cin >> a;
// skriv ut objektet till stdout
std::cout << a << std::endl;
```

Iteratorer

In- och
utmatningsoperatorer

In- och utmatningsiteratörer

- STL-behållare kan man fylla direkt från en ström med en inmatnings-iterator, **std::istream_iterator**
- Man kan mata ut innehållet i en behållare till en ström med en utmatnings-iterator, **std::ostream_iterator**

In- och utmatningsiteratörer

```
std::vector<int> v;  
  
// hämta från stdin tills EOF-indikator, lägg sist  
std::copy(std::istream_iterator<int>(std::cin),  
          std::istream_iterator<int>(), // EOF  
          std::back_inserter(v));  
  
std::transform(v.begin(), // beräkna och skriv ut x % 10  
              v.end(), // x % y, y = 10  
              std::ostream_iterator<int>(std::cout, "\n"),  
              std::bind2nd(std::modulus<int>(), 10));
```

Filhantering

Öppning
Läsning
Skrivning
Accesstyper

Filhantering

- För att knyta en ström till en fil använder man en filström
- **std::ifstream** ger inmatning och tar ett filnamn som argument
- **std::ofstream** ger utmatning
- **std::fstream** ger en generell filström som klarar att både läsa och skriva

Filhantering

```
#include <fstream>           // ifstream, ofstream

std::ifstream input("my_file.in"); // skapa infil
std::ofstream output("my_file.out"); // skapa utfil

// kontrollera att filerna gick att öppna
if(!input)
    std::cout << "open failed for input file";
if(!output)
    std::cout << "open failed for output file";

std::vector<int> v;
int i;
input >> i;           // hämta data
v.push_back(i);       // lägg i vektorn

output << v.back() << std::endl; // skriv element
```

Filhantering

Man kan bestämma hur en fil ska öppnas (lägg till, skriv, läs)

```
#include <iostream> // ios_base
#include <fstream>  // ofstream, fstream

std::ofstream s1("result", std::ios_base::app);
std::fstream s2("temp", std::ios_base::in |
                  std::ios_base::out);
```

Filhantering

Olika åtkomsttyper:

- **app** - lägg till (*append*)
- **ate** - finn slutet (*at end*)
- **binary** - binär
- **in** - läs
- **out** - skriv
- **trunc** - töm fil (*truncate*)